



# Plagiarism Checker X - Report

Originality Assessment

0%

Overall Similarity

**Date:** May 23, 2026 (03:10 AM)

**Matches:** 16 / 4977 words

**Sources:** 1

**Remarks:** No similarity found,  
your document looks healthy.

**Verify Report:**

Scan this QR Code



## INDEX

Chapter No.

Details

Page No.

1.

Introduction

4-7

1.1.

Company Profile

4

1.2.

Problem Statement

4

1.3.

Existing System and Need for the System

5

1.4.

Scope of the Proposed System

5

1.5.

Objectives of the Proposed System

6

1.6.

Operating Environment — Hardware and Software

6

1.7.

Technology Used

7

2.

System Analysis and Design

8-20

2.1.

System Requirements (Functional and Non-Functional)

9

2.2.

Feasibility Study (Technical, Economic, Operational)

10

2.3.

Architecture Design

11

2.4.

1 Entity Relationship Diagram (ERD)

12

2.5.

Class Diagram

13

2.6.

Use Case Diagram

14

2.7.

Activity Diagram

15

2.8.

Sequence Diagram

16

2.9.

## Component Diagram

17

2.10.

## Module Hierarchy Diagram

18

2.11.

## Table Design

19

2.12.

## Data Dictionary

20

3.

## Implementation

21-27

3.1.

## Input Screens

22

3.2.

## Output Screens / Reports

23

3.3.

## Sample Code

26

4.

## Testing

28-31

4.1.

## Test Strategy

28

4.2.

Unit Test Plan

29

4.3.

Acceptance of Test Plan

30

4.4.

Test Cases

30

4.5.

Results

31

5.

Conclusion

32-33

5.1.

Summary / Conclusion

32

5.2.

Limitations of the System

32

5.3.

Future Enhancements

33

6.

References / Bibliography

34

7.

Appendices

35-36

7.1.

Annexure I — Additional Screens

35

7.2.

Annexure II — Progress Sheet

36

## LIST OF FIGURES

Figure No.

Caption

Page No.

Fig. 1

ETL + ML Pipeline Architecture

10

Fig. 2

System Architecture — 4-Layer Design

11

Fig. 3

1 Entity Relationship Diagram (ERD)

12

Fig. 4

Class Diagram — Module Structure

13

Fig. 5

Use Case Diagram

14

Fig. 6

Activity Diagram — ETL Pipeline Flow

15

Fig. 7

Sequence Diagram — Login and Prediction

16

Fig. 8

Component Diagram — System Dependencies

17

Fig. 9

Module Hierarchy Diagram

18

## LIST OF TABLES

Table No.

Caption

Page No.

Table 1

Hardware Requirements

6

Table 2

Software Requirements

7

Table 3

Technology Stack

7

Table 4

## Functional Requirements

8

### Table 5

## Non-Functional Requirements

9

### Table 6

### customer\_churn\_raw — Table Design

19

### Table 7

### customer\_churn\_cleaned — Table Design

19

### Table 8

### Data Dictionary — Key Fields

20

### Table 9

### Unit Test Cases — ETL Pipeline

28

### Table 10

### Unit Test Cases — ML and Auth

29

### Table 11

### Acceptance Test Plan

30

### Table 12

### Test Results Summary

31

## Chapter 1: Introduction



## 1.1 Company Profile

D-Table Analytics is a technology and analytics company that provides solutions in data analytics, business intelligence, software development, and process automation. The organization focuses on helping businesses transform raw data into meaningful insights for better decision-making and operational efficiency. The company develops customized web applications, reporting systems, dashboards, and automation tools tailored to business requirements.

D-Table Analytics emphasizes practical and scalable technology solutions using modern tools and emerging technologies such as Artificial Intelligence and cloud-based systems. The company works with businesses to improve workflow management, data handling, and digital transformation processes through innovative and data-driven approaches.

## 1.2 Problem Statement

Telecom companies experience significant revenue loss due to customer churn — the phenomenon where customers discontinue services and switch to competitors. Analysis of the IBM Telco Customer Churn dataset reveals a churn rate of approximately 26.5%, which translates to substantial monthly revenue loss. The core challenges driving this problem are:

- ❑ Raw customer data is inconsistent and contains missing values that make direct analysis unreliable.
- ❑ Manual identification of at-risk customers is slow, subjective, and does not scale with growing data volumes.
- ❑ Decision-makers lack real-time, interactive dashboards to monitor churn patterns and act proactively.
- ❑ No automated mechanism exists to predict individual customer churn probability or recommend personalised retention interventions.
- ❑ Existing data processing tools cannot handle the volume and velocity of enterprise-scale customer datasets.

### 1.3 Existing System and Need for the System

The existing system in the target organisation relies on periodic manual reporting using spreadsheet tools. Customer service teams identify at-risk customers based on billing complaints and support ticket volumes — an approach with serious limitations:

- ❑ No real-time or near-real-time churn prediction capability.
- ❑ Analysis limited to aggregated monthly reports with a 2 to 4 week lag.
- ❑ No personalised retention offer generation based on individual customer profiles.
- ❑ No secure role-based access control for analytical tools.
- ❑ Not scalable — manual processes break down beyond a few thousand records.

The proposed system directly addresses all these limitations through an automated, scalable ETL pipeline integrated with machine learning and an interactive Streamlit web dashboard secured by role-based authentication.

### 1.4 Scope of the Proposed System

The Telecom Customer Churn Analytics Platform covers the following functional scope:

- ❑ End-to-end ETL pipeline: CSV extraction, data cleaning, feature engineering, and loading into a PostgreSQL relational database.
- ❑ Machine learning pipeline: training of three classification models (Random Forest, Gradient Boosting, Logistic Regression), automated best-model selection by ROC-AUC score, and batch scoring of all customers.
- ❑ Retention offer recommendation engine with seven distinct offer strategies mapped to customer risk profiles.
- ❑ Streamlit dashboard with five pages: Executive Dashboard, Churn Predictor, At-Risk Customers, Model Performance, and Admin Panel.
- ❑ Authentication system: PBKDF2-SHA256 password hashing, session management via `st.session_state`, and complete access logging to CSV.
- ❑ PySpark integration demonstrating enterprise-scale distributed processing capability.
- ❑ Comprehensive test suite: 35 unit tests across 9 test classes covering all modules.
- ❑ Four Jupyter notebooks documenting the complete data science workflow.

## 1.5 Objectives of the Proposed System

- Design and implement a complete ETL pipeline using Python, Pandas, and PySpark.
- Clean, transform, and engineer features from the IBM Telco Customer Churn dataset.
- Train and evaluate multiple ML models; automatically select and save the best by ROC-AUC score.
- Score all customers with ChurnProbability and ChurnRisk classification.
- Generate personalised retention offers based on each customer's risk profile.
- Develop a secure, role-based Streamlit analytics dashboard.
- Implement enterprise-grade authentication with session management and access audit logging.
- Demonstrate scalable distributed data processing using Apache PySpark.
- Achieve 100%-unit test pass rate across all ETL, ML, and auth modules.

## 1.6 Operating Environment — Hardware and Software

### Hardware Requirements

Table 1: Hardware Requirements

| Component        | Minimum Specification                                           |
|------------------|-----------------------------------------------------------------|
| Processor        | Intel Core i5 / AMD Ryzen 5, 2.0 GHz+                           |
| RAM              | 8 GB minimum; 16 GB recommended for PySpark                     |
| Storage          | 50 GB free disk space (SSD preferred)                           |
| Operating System | Windows 10/11, Ubuntu 20.04+, or macOS 12+                      |
| Network          | Internet connection for package installation and Kaggle dataset |

## Software Requirements

Table 2: Software Requirements

Software

Version

Purpose

Python

3.11+

Core programming language

PostgreSQL

15+

Relational database

Apache PySpark

3.5

Distributed data processing

Streamlit

1.35+

Web dashboard framework

scikit-learn

1.4+

Machine learning models

Pandas

2.0+

Data manipulation and analysis

Plotly

5.20+

Interactive visualisation charts

Jupyter Notebook

Latest

EDA and documentation

PyCharm

Latest

Development IDE

Git / GitHub

Latest

Version control

pytest

8.0+

Unit testing framework

1.7 Technology Used

Table 3: Technology Stack

Component

Technology / Framework

Frontend / Dashboard

Streamlit 1.35 (Python-based web UI framework)

Visualisation

Plotly Express and Plotly Graph Objects

Backend Language

Python 3.11

Data Processing

Pandas 2.0, NumPy

Big Data Processing

Apache PySpark 3.5 (local and cluster mode)

Machine Learning

scikit-learn 1.4, XGBoost 2.0

Database

PostgreSQL 15 via SQLAlchemy 2.0

Authentication

PBKDF2-SHA256 (Python hashlib), st.session\_state

Testing

pytest 8.0, pytest-cov

Dataset

IBM Telco Customer Churn (Kaggle, 7,043 records)

Version Control

Git and GitHub

Notebook Environment

Jupyter Notebook

Chapter 2: System Analysis and Design

2.1 System Requirements

Functional Requirements

The functional requirements define the core system behaviour and features supported by the Telecom Churn Analytics Platform. These requirements describe ETL processing, machine learning operations, recommendation generation, dashboard functionality, and authentication mechanisms.

Table 4: Functional Requirements

FR ID

Module

Requirement Description

FR-01

ETL

System shall extract data from the IBM Telco CSV and validate all 21 required columns.

FR-02

ETL

System shall clean data: fix TotalCharges type, impute median, and remove duplicates.

FR-03

ETL

System shall engineer features: tenure\_group, RevenueCategory, AnnualRevenue, LoyaltyScore.

FR-04

ETL

System shall load cleaned and scored data to PostgreSQL in two tables.

FR-05

ML

System shall train three ML models and auto-select the best by ROC-AUC.

FR-06

ML

System shall score all customers with ChurnProbability and ChurnRisk (High/Medium/Low).

FR-07

Recommend

System shall generate personalised retention offers for at-risk customers.

FR-08

Dashboard

System shall display KPIs, charts, and at-risk lists across 5 Streamlit pages.

FR-09

Auth

System shall authenticate users with PBKDF2-SHA256 hashed passwords.

FR-10

Admin

Admin shall add/delete users, change passwords, and view complete access log.

## Non-Functional Requirements

The non-functional requirements specify system quality attributes including performance, scalability, security, reliability, usability, portability, and maintainability constraints.

Table 5: Non-Functional Requirements

| Category        | Requirement                                                                               |
|-----------------|-------------------------------------------------------------------------------------------|
| Performance     | ETL pipeline for 7,043 records shall complete within 60 seconds on standard hardware.     |
| Scalability     | PySpark pipeline shall support datasets exceeding 1 million records in cluster mode.      |
| Security        | Passwords hashed using PBKDF2-SHA256 with 260,000 iterations; never stored in plain text. |
| Reliability     | System shall log all login attempts, page views, predictions, and admin actions to CSV.   |
| Usability       | Dashboard navigable by non-technical business users with sidebar filters and KPI cards.   |
| Portability     | System runs on Windows 10/11, Ubuntu 20.04+, and macOS 12+ with Python 3.11+.             |
| Maintainability | Modular architecture with 9 test classes and 35 unit tests covering all modules.          |

## 2.2 Feasibility Study

### Technical Feasibility

The project uses a mature, production-grade open-source technology stack. Python 3.11, Pandas 2.0, PySpark 3.5, scikit-learn 1.4, Streamlit 1.35, and PostgreSQL 15 all have extensive documentation, active communities, and long-term support. The IBM Telco dataset (7,043 records, 21 columns) is well within the processing capability of the stack on standard academic hardware. The PySpark integration demonstrates that the architecture



can scale to enterprise datasets exceeding one million records.

### Economic Feasibility

The project is developed entirely using open-source, freely licensed software. There are no licensing costs for Python, PostgreSQL, Streamlit, scikit-learn, Jupyter, or any other component. Hardware requirements (8 GB RAM, modern processor) are standard in academic and professional environments. The total cost is limited to hardware usage and internet connectivity for package installation and dataset download.

### Operational Feasibility

The Streamlit dashboard is designed for non-technical business analysts. Navigation uses a clear sidebar with labelled pages and interactive filters. Role-based access control restricts sensitive functions to authorised administrators. Maintenance requires only running the ETL pipeline to refresh data — the dashboard updates automatically when new scored data is available.

## 2.3 Architecture Design

The system follows a four-layer architecture. The Presentation Layer (Streamlit) handles all user interaction. The Application Layer contains business logic modules (extract, transform, recommend, auth). The ETL and ML Processing Layer contains the training, prediction, and loading modules. The Data Storage Layer holds raw CSV, PostgreSQL tables, model .pkl files, and access logs.

Figure 1 — ETL + ML Pipeline Architecture

Figure 2 — System Architecture (4-Layer Design)

## 2.4 Entity Relationship Diagram (ERD)

The system uses PostgreSQL as the primary analytical store with two tables. The `customer_churn_raw` table preserves the original extracted data as an audit trail. The

customer\_churn\_cleaned table stores transformed and scored data used by the dashboard. User credentials are stored in users.json and the access log is maintained in logs/access\_log.csv.

Figure 3 — 1 Entity Relationship Diagram (ERD)

## 2.5 Class Diagram

The system comprises eight core classes, each encapsulating a distinct concern of the ETL-ML pipeline. DataExtractor handles CSV reading and validation. DataTransformer manages cleaning and feature engineering. DataLoader writes to PostgreSQL. ModelTrainer handles ML training and artifact persistence. ChurnPredictor scores customers. OfferEngine generates retention recommendations. AuthManager handles authentication and logging. SparkETL provides distributed processing capability.

Figure 4 — Class Diagram: Module Structure

## 2.6 Use Case Diagram

Two primary actors interact with the system. The User actor can log in, view the Executive Dashboard, run the Churn Predictor, view At-Risk Customers, and export reports. The Admin actor has all user capabilities plus access to Model Performance, User Management, Access Log, Password Management, and ETL pipeline execution.

Figure 5 — Use Case Diagram

## 2.7 Activity Diagram

The activity diagram illustrates the sequential flow of control through the ETL pipeline. The process begins with CSV extraction, proceeds through column validation (with a decision point for error handling), data backup, cleaning, feature engineering, ML model training,

customer scoring, offer generation, and finally PostgreSQL loading. The PySpark path runs as a parallel distributed alternative to the Pandas pipeline.

Figure 6 — Activity Diagram: ETL Pipeline Flow

## 2.8 Sequence Diagram

The sequence diagram shows message flow between system components during two key interactions. The Login Sequence shows credentials being passed from app.py to auth.py, where PBKDF2 verification occurs, followed by a successful login event being written to the access log and a session redirect. The Prediction Sequence shows how a customer form submission triggers `predict_single()`, loads the `.pkl` model, runs `predict_proba()`, generates a retention offer, logs the prediction event to PostgreSQL, and returns the gauge chart and offer card to the user.

Figure 7 — Sequence Diagram: Login and Prediction

## 2.9 Component Diagram

The component diagram illustrates all software components and their dependencies across three rows: the processing layer (CSV through extract, transform, train), the logic layer (load, predict, recommend, auth, app), and the storage layer (raw CSV, PostgreSQL, model files, user store, access log). Each component is represented with the standard UML notch symbol indicating an independent deployable unit.

Figure 8 — Component Diagram: System Dependencies

## 2.10 Module Hierarchy Diagram

The module hierarchy organises all system components under four top-level subsystems: ETL Layer (extract, transform, load), ML Layer (train\_model, predict, recommend), App Layer (app.py, spark\_etl), and Auth Layer (auth, utils). Each subsystem is further

decomposed to show individual functions and methods, providing a complete structural view of the codebase.

Figure 9 — Module Hierarchy Diagram

2.11 Table Design

Table 6: customer\_churn\_raw — Table Design

| Column Name                            |
|----------------------------------------|
| Data Type                              |
| Constraint                             |
| Description                            |
| customerID                             |
| VARCHAR(50)                            |
| PRIMARY KEY                            |
| Unique 7-character customer identifier |
| gender                                 |
| VARCHAR(10)                            |
| NULL                                   |
| Male or Female                         |
| SeniorCitizen                          |
| SMALLINT                               |
| NULL                                   |
| 0 = No, 1 = Yes                        |
| tenure                                 |
| INT                                    |
| NULL                                   |
| Months with the company                |
| MonthlyCharges                         |

Float

Null

Monthly billing amount

TotalCharges

Float

Null

Cumulative charges (blank for new)

Contract

VarChar(50)

Null

Month-to-month, One year, Two year

PaymentMethod

VarChar(100)

Null

Payment method used

InternetService

VarChar(50)

Null

DSL, Fiber optic, or No

Churn

VarChar(10)

Null

Yes or No (raw label)

Table 7: customer\_churn\_cleaned — Table Design

Column Name

Data Type

Constraint

Description

customerID

VARCHAR(50)

PRIMARY KEY

Unique customer identifier

Churn

SMALLINT

NOT NULL

1 = Churned, 0 = Retained

tenure

INT

NULL

Months with the company

MonthlyCharges

FLOAT

NULL

Monthly billing amount

TotalCharges

FLOAT

NULL

Median-imputed cumulative charges

tenure\_group

VARCHAR(20)

NULL

0-1yr, 1-2yr, 2-4yr, 4-6yr

RevenueCategory

VARCHAR(10)

NULL

Low, Medium, or High

AnnualRevenue

FLOAT

NULL

MonthlyCharges multiplied by 12

LoyaltyScore

FLOAT

NULL

Normalised tenure: tenure / max\_tenure

ChurnProbability

FLOAT

NULL

ML predicted churn probability 0.0 to 1.0

ChurnRisk

VARCHAR(10)

NULL

Low, Medium, or High risk tier

OfferTitle

VARCHAR(100)

NULL

Recommended retention offer title

Contract

VARCHAR(50)

NULL

Preserved from OHE for dashboard

InternetService

VARCHAR(50)

NULL

Preserved from OHE for dashboard

## 2.12 Data Dictionary

Table 8: Data Dictionary — Key Fields

Field Name

Type

Range / Values

Description

customerID

String

Unique

7-character alphanumeric ID assigned to each customer

tenure

Integer

0 to 72

Number of months the customer has been with the company

MonthlyCharges

Float

18.25 to 118.75

Amount charged to the customer monthly in USD

TotalCharges

Float

18.85 to 8684.8

Total amount charged; blank entries imputed with median

Churn

Binary Int

0 or 1

Target variable: 1 = churned, 0 = retained

ChurnProbability

Float



0.0 to 1.0

ML-predicted probability of churn for each customer

ChurnRisk

String

Low/Med/High

Risk tier: Low<35%, Medium 35-65%, High>65%

LoyaltyScore

Float

0.0 to 1.0

Normalised tenure: tenure divided by maximum tenure

AnnualRevenue

Float

219 to 1425

Projected annual revenue: MonthlyCharges multiplied by 12

RevenueCategory

String

Low/Med/High

Low < Rs.35, Medium Rs.35-70, High > Rs.70 monthly

tenure\_group

String

4 categories

0-1yr, 1-2yr, 2-4yr, 4-6yr based on tenure months

gender\_encoded

Binary Int

0 or 1

Male = 1, Female = 0

OfferTitle

String

7 offer types

Recommended retention offer for each at-risk customer

password\_hash

Text

salt:key

PBKDF2-SHA256 hash stored as salt:hex\_key string

## Chapter 3: Implementation

### 3.1 Input Screens

The system accepts all user input through the Streamlit web interface. The primary input points are:

- ☐ Login Screen: username and password fields with a Sign In button. Displays an error for invalid credentials and redirects to the dashboard on success.
- ☐ Churn Predictor Form: 12 input fields including tenure, contract type, payment method, internet service, charges, customer demographics, and billing preferences with appropriate UI controls.
- ☐ Admin Panel — Add User: username, full name, password (masked), and role dropdown. Validates that username is unique and password meets minimum length.
- ☐ Admin Panel — Change Password: user selector dropdown, new password, and confirmation fields with mismatch validation.

All numeric input fields use type-safe Streamlit widgets that prevent invalid data entry.

String inputs are validated server-side in auth.py and the recommendation engine before processing.

### 3.2 Output Screens / Reports

The system produces the following output screens and reports, each containing at least five valid data records:

- Executive Dashboard: Dashboard includes six KPI cards and multiple visualizations for churn distribution, contract types, internet services, payment methods, monthly charges, and revenue-at-risk analysis.

- Churn Predictor Output: Displays a Plotly gauge chart for churn probability, a color-coded risk indicator, and a personalized retention offer with discount details and recommended action.

- At-Risk Customers Report: filterable and sortable data table with columns: customerID, ChurnProbability, ChurnRisk, Contract, InternetService, PaymentMethod, MonthlyCharges, tenure, OfferTitle, OfferDiscount. Includes pie chart of risk distribution and bar chart of offer type frequencies. Exportable to CSV.

- Access Log Report: timestamped table of all system events - login attempts (success/fail), page views, prediction runs, user additions, and deletions. Filterable by event type and username. Full log downloadable as CSV.

- Model Performance Report: comparison table of three models (Accuracy, Precision, Recall, F1, ROC-AUC), confusion matrix heatmap, feature importance horizontal bar chart, and ROC-AUC bar chart with best model highlighted.

### 3.3 Sample Code

extract.py — Phase 1: Data Extraction

```
def extract_data(filepath: str = RAW_DATA) -> pd.DataFrame:
```

```
"""
```

```
Extract customer churn dataset from CSV file.
```

```
"""
```

```
log("EXTRACT", f"Reading CSV: {filepath}")
```

```
# Validate file existence
```

```
if not os.path.exists(filepath):
```

```
    raise FileNotFoundError(
```

```
        f"Dataset not found: {filepath}"
```

```
    )
```

```
# Load dataset
```

```
df = pd.read_csv(filepath)
```

```
log(
```

```
    "EXTRACT",
```

```
    f"Shape: {df.shape[0]:,} rows x "
```

```
    f"{df.shape[1]} columns"
```

```
)
```

```
# Validate required schema
```

```
validate_columns(df, REQUIRED_COLUMNS)
```

```
# Create backup copy
```

```
shutil.copy2(filepath, BACKUP_DATA)
```

```
return dfreturn df
```

transform.py — Feature Engineering

```
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
```

```
    # Create annual revenue feature
```

```
    df["AnnualRevenue"] = (
```

```
        df["MonthlyCharges"] * 12
```

```
    )
```

```

# Generate loyalty score
df["LoyaltyScore"] = (
    df["tenure"] / df["tenure"].max()
).round(3)

# Revenue category conditions
conditions = [
    df["MonthlyCharges"] < 35,
    (df["MonthlyCharges"] >= 35) &
    (df["MonthlyCharges"] < 70),
    df["MonthlyCharges"] >= 70
]

# Assign revenue category
df["RevenueCategory"] = np.select(
    conditions,
    ["Low", "Medium", "High"],
    default="Low"
).astype(str)

return df

```

train\_model.py — ML Model Training

```

models = {
    "RandomForest": RandomForestClassifier(n_estimators=200, max_de
pth=8, class_weight="balanced"),
    "GradientBoosting": GradientBoostingClassifier(n_estimators=150, learni
ng_rate=0.08),
    "LogisticRegression": Pipeline([("scaler", StandardScaler()), ("clf", LogisticR
egression(max_iter=1000))]), }
for name, model in models.items():
    model.fit(X_train, y_train)
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[: ,1])
    if auc > best_auc:
        best_auc=auc; best_model=model; best_name=name

```

## auth.py — Password Hashing and Verification

```
def _hash_password(password: str) -> str:    salt = os.urandom(32).hex()    key =  
hashlib.pbkdf2_hmac(        "sha256", password.encode(), salt.encode(), 260_000)  
return f"{salt}:{key.hex()}"  
def _verify_password(password: str, stored_hash: str) -> bool:  
salt, key_hex = stored_hash.split(":", 1)    key = hashlib.pbkdf2_hmac(        "sha256",  
password.encode(), salt.encode(), 260_000)    return key.hex() == key_hex
```

## recommend.py — Retention Offer Engine

```
def recommend_offer(row: pd.Series) -> dict:    prob = float(row.get("ChurnProbability",  
0.5))    risk = str(row.get("ChurnRisk", "Medium"))    if prob >= 0.80:        return  
OFFERS["RETENTION_CALL"]    if row.get("SeniorCitizen") == 1 and risk in  
["High", "Medium"]:        return OFFERS["SENIOR_PLAN"]    if "Month-to-month" in  
str(row.get("Contract", "")) and risk=="High":        return  
OFFERS["CONTRACT_UPGRADE"]    if "Fiber" in str(row.get("InternetService", "")) and  
risk=="High":        return OFFERS["TECHSUPPORT_BUNDLE"]    if "Electronic check" in  
str(row.get("PaymentMethod", "")):        return OFFERS["AUTOPAY_CASHBACK"]  
return OFFERS["STANDARD_LOYALTY"]
```

## Chapter 4: Testing

### 4.1 Test Strategy

The testing strategy for the Telecom Churn Analytics Platform covers three levels of testing:

- Unit Testing: Individual functions within each module are tested in isolation using pytest. Each module contains dedicated test classes with multiple automated test cases covering expected behaviour, validation rules, boundary conditions, and error handling. The automated test suite implemented in tests/test\_etl.py contains 9 test classes and 48 test cases.
- Integration Testing: The complete ETL and Machine Learning pipeline is tested end-to-end by executing the pipeline workflow and verifying that the generated churn\_scored.csv file contains expected columns, correct datatypes, valid probability ranges, and non-null

recommendation outputs.

□ User Acceptance Testing: The Streamlit dashboard is validated using real scored customer data by testing dashboard pages, KPI rendering, filtering functionality, churn prediction outputs, recommendation logic, and CSV export functionality.

Tests are executed using:

```
python -m pytest tests/ -v --tb=short --cov=src
```

## 4.2 Unit Test Plan

Table 9: Unit Test Cases — ETL Pipeline

Test ID

Test Case

Input / Condition

Expected Result

TC-E01

Validate columns — pass

All required columns

No exception raised

TC-E02

Validate columns — fail

Missing column

ValueError raised

TC-E03

Dataset shape correct

Sample DataFrame

shape[1] == 21

TC-E04

CustomerID unique

Sample DataFrame

Unique IDs only

TC-E05

Dataset not empty

Loaded DataFrame

`len(df) > 0`

TC-C01

TotalCharges numeric

Cleaned DataFrame

`dtype == float64`

TC-C02

No null TotalCharges

Raw blank values

No nulls remain

TC-C03

No duplicate rows

Cleaned DataFrame

0 duplicates

TC-C04

Strings stripped

Object columns

No extra spaces

TC-C05

Row count preserved

No duplicate sample

Counts preserved

TC-T01

Churn binary encoded

Transformed DataFrame

Values in `{0,1}`



TC-T02

Contract preserved

After encoding

Column exists

TC-T03

InternetService preserved

After encoding

Column exists

TC-T04

Tenure group valid

Grouped DataFrame

Valid labels only

Table 10: Unit Test Cases — ML and Auth Modules

Test ID

Test Case

Input / Condition

Expected Result

TC-F01

AnnualRevenue calculation

Featured DataFrame

Monthly \* 12

TC-F02

LoyaltyScore range

Featured DataFrame

$0 \leq \text{score} \leq 1$

TC-V01

Validation passes

Clean DataFrame

Returns True

TC-V02

Validation fails on nulls

DataFrame with NaN

Returns False

TC-P01

Predict returns dictionary

Customer input

Valid keys returned

TC-P02

Probability range valid

Prediction output

$0.0 \leq p \leq 1.0$

TC-R01

High-risk contract offer

High-risk customer

Offer generated

TC-R02

Retention call generated

Very high-risk customer

RETENTION\_CALL

TC-R03

Offer fields present

Recommendation output

Required fields exist

TC-A01

Password hash verify

Plain password

Verification succeeds

TC-A02

Wrong password fails

Incorrect password

Verification fails

TC-A03

Add/delete user

testuser

User added then removed

#### 4.3 Acceptance of Test Plan

Table 11: Acceptance Test Plan

AT ID

Test Scenario

Acceptance Criteria

Result

AT-01

Login with valid credentials

Dashboard loads

Pass

AT-02

Wrong password login

Access denied

Pass

AT-03

Run ETL pipeline

CSV generated

Pass

AT-04

Dashboard loads correctly

KPIs displayed

Pass

AT-05

High-risk prediction

Retention offer generated

Pass

AT-06

Filter high-risk users

Only matching rows shown

Pass

AT-07

Export CSV

Download successful

Pass

AT-08

Admin adds user

User added successfully

Pass

AT-09

Role restriction works

Access denied shown

Pass

AT-10

Logs recorded

Timestamp entries exist

Pass

4.4 Test Cases

The complete automated test suite implemented in tests/test\_etl.py contains 48 test cases across 9 test classes. TestExtract (5 tests) validates column presence, dataset shape, and customer ID uniqueness. TestCleaning (6 tests) verifies TotalCharges type conversion, null handling, duplicate removal, and string stripping. TestTransform (10 tests) checks binary encoding, One-Hot Encoding (OHE) preservation of Contract, InternetService, and PaymentMethod columns, along with tenure group validation. TestFeatureEngineering (7 tests) confirms AnnualRevenue calculation, LoyaltyScore range validation, and null-free engineered features. TestValidation (3 tests) verifies data validation logic. TestPredict (3 tests) validates machine learning output format and probability bounds. TestRecommend (7 tests) validates business recommendation rules and offer assignment logic. TestAuth (4 tests) tests password hashing and user CRUD operations. TestConfig (3 tests) validates configuration imports and datatype consistency.

4.5 Results

Table 12: Test Results Summary

Test Class

Total Tests

Passed

Skipped

Failed

TestExtract

5

5

0

0

TestCleaning

6

6

0

0

TestTransform

10

10

0

0

TestFeatureEngineering

7

7

0

0

TestValidation

3

3

0

0

TestPredict

3

3

0

0

TestRecommend

7

7

|            |
|------------|
| 0          |
| 0          |
| TestAuth   |
| 4          |
| 4          |
| 0          |
| 0          |
| TestConfig |
| 3          |
| 3          |
| 0          |
| 0          |
| TOTAL      |
| 48         |
| 48         |
| 0          |
| 0          |

All 48-unit tests pass after running the complete ETL + ML pipeline. The 100% pass rate demonstrates correctness of all data processing, ML inference, offer recommendation, and authentication modules.

Three compatibility warnings related to future Pandas datatype behaviour were observed during testing. These warnings do not affect current system functionality.

## Chapter 5: Conclusion

### 5.1 Summary / Conclusion

This project successfully demonstrates the design and implementation of a complete, production-grade ETL pipeline for Telecom Customer Churn Analysis. The system

integrates data engineering, machine learning, and business intelligence into a unified deployable platform built entirely on open-source technologies.

Key achievements include a complete ETL pipeline processing 7,043 customer records; a machine learning pipeline training three models with automated best-model selection; churn probability scoring for all customers with three-tier risk classification; a seven-strategy personalised retention offer recommendation engine; a Streamlit analytics dashboard with five pages and role-based access; a secure PBKDF2-SHA256 authentication system with complete audit logging; PySpark integration for enterprise-scale distributed processing; and a comprehensive test suite achieving a 100% pass rate across 35 unit tests.

The project demonstrates that modern open-source tools — Python, Pandas, PySpark, scikit-learn, and Streamlit — can be integrated into a cohesive, scalable, and deployable analytics platform suitable for real-world telecom customer retention use cases. The architecture is modular and extensible, making it straightforward to add new data sources, ML models, or dashboard pages.

## 5.2 Limitations of the System

- The system uses the IBM Telco sample dataset (7,043 records), which may not fully represent all telecom customer segments, regional billing patterns, or enterprise-scale data volumes.
- The retention offer recommendation engine uses rule-based logic. A collaborative filtering or reinforcement learning approach would generate more granular, personalised recommendations.
- User credentials are stored in a flat JSON file. A dedicated users table in PostgreSQL would be more robust and auditable at enterprise scale.
- The Streamlit dashboard does not support real-time data ingestion. The ETL pipeline must be re-run manually to refresh customer scores and recommendations.
- PySpark runs in local mode in the current implementation. A distributed cluster deployment (AWS EMR, Databricks, or Azure HDInsight) is required for true enterprise



scalability.

- The system currently has no email or SMS notification capability to alert the retention team when a customer crosses the high-risk threshold.

### 5.3 Future Enhancements

- Apache Airflow integration: automate ETL pipeline scheduling using DAGs for daily or weekly data refresh without manual intervention.
- Real-time streaming ETL: integrate Apache Kafka for real-time customer event ingestion and immediate churn risk recalculation.
- Advanced ML models: implement neural networks (TensorFlow or PyTorch) and AutoML for improved prediction accuracy.
- Cloud deployment: host the pipeline on AWS (S3 + RDS + EC2) or Azure (Blob Storage + Azure SQL + App Service) for production availability.
- SHAP explainability: integrate SHAP values to provide human-readable explanations for each individual churn prediction.
- REST API layer: expose prediction and recommendation endpoints via FastAPI for integration with CRM and billing systems.
- Power BI integration: export scored data to Power BI for enterprise-grade business intelligence and executive reporting.
- Snowflake data warehouse: migrate the analytical storage layer to Snowflake for columnar querying and seamless BI tool integration.

## Chapter 6: References / Bibliography

[1] IBM Sample Datasets. (2024). Telco Customer Churn Dataset. Kaggle.

<https://www.kaggle.com/blatchar/telco-customer-churn>

[2] Apache Software Foundation. (2024). PySpark API Documentation — Apache Spark 3.5. <https://spark.apache.org/docs/latest/api/python/>

[3] Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io>

[4] scikit-learn Developers. (2024). scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

- [5] McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media.
- [6] SQLAlchemy Authors. (2024). SQLAlchemy Documentation.  
<https://docs.sqlalchemy.org>
- [7] PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation.  
<https://www.postgresql.org/docs/15/>
- [8] Chen, T., and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016, 785-794.
- [9] Plotly Technologies Inc. (2024). Plotly Python Open-Source Graphing Library.  
<https://plotly.com/python/>
- [10] pytest Development Team. (2024). pytest Documentation. <https://docs.pytest.org>
- [11] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- [12] Provost, F., and Fawcett, T. (2013). Data Science for Business. O'Reilly Media.

## Chapter 7: Appendices

### 7.1 Annexure I — Additional Input and Output Screens

The following additional screens are produced by the system and submitted as part of project documentation:

- ☐ Screen A1: Streamlit login page with username and password fields, Sign In button, and footer.
- ☐ Screen A2: Executive Dashboard — KPI row showing Total Customers (7,043), Churned (1,869), Churn Rate (26.5%), Revenue Lost per Month, Average Tenure, and High-Risk count.
- ☐ Screen A3: Churn Predictor form filled with a high-risk customer profile — Month-to-month contract, Electronic Check, Fiber optic, tenure 2 months, monthly charges Rs. 89.
- ☐ Screen A4: Churn Predictor output — gauge at 87%, HIGH RISK badge, Retention Call offer card displayed.
- ☐ Screen A5: At-Risk Customers page filtered to High risk, showing 10 records with ChurnProbability, Contract, OfferTitle, and MonthlyCharges.
- ☐ Screen A6: Model Performance page showing model comparison table and feature

importance chart with tenure and MonthlyCharges as top drivers.

❑ Screen A7: Admin Panel — User Accounts table showing Div-dev (Admin) and at least two regular user accounts with created dates.

❑ Screen A8: Admin Panel — Access Log table showing at least 10 recent events including login, page views, and prediction runs.

All screens were captured from the running Streamlit application with valid data loaded from the ETL pipeline output (churn\_scored.csv, 7,043 records).

## 7.2 Annexure II — Progress Sheet

Phase

Task Description

Status

Week

1

Project proposal, literature review, and problem definition

Completed

1

2

Environment setup: Python venv, PostgreSQL, VS Code, Git

Completed

2

3

Dataset acquisition from Kaggle and initial EDA (Notebook 01)

Completed

2

4

ETL Phase 1: extract.py — CSV reading, validation, backup

Completed

3

5

ETL Phase 2: transform.py — cleaning, encoding, feature engineering

Completed

4

6

ETL Phase 3: load.py — PostgreSQL schema and data loading

Completed

4

7

PySpark integration: spark\_etl.py and distributed pipeline

Completed

5

8

ML pipeline: train\_model.py — RF, GBT, LR training and selection

Completed

6

9

Prediction and offer modules: predict.py, recommend.py

Completed

7

10

Streamlit dashboard: app.py — 5 pages with Plotly charts

Completed

8-9

11

Authentication system: auth.py, users.json, access\_log.csv

Completed

10

12

Unit testing: test\_etl.py — 9 classes, 35 test cases, 100% pass

Completed

11

13

Jupyter notebooks 01-04: EDA, cleaning, features, visualisation

Completed

11

14

UML diagrams (9 figures) and project report writing

Completed

12

15

Final review, screenshot capture, and submission preparation

Completed

13

Telecom Customer Churn Analysis — ETL, ML and Streamlit Dashboard

IIMS Chinchwad, Pune | MCA Sem IV | 2025-26

## Sources

|   |                                                                                                                                                              |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <a href="https://dbschema.com/blog/design/entity-relationship-diagram/">https://dbschema.com/blog/design/entity-relationship-diagram/</a><br>INTERNET<br><1% |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

|                        |     |
|------------------------|-----|
| EXCLUDE CUSTOM MATCHES | ON  |
| EXCLUDE QUOTES         | OFF |
| EXCLUDE BIBLIOGRAPHY   | OFF |